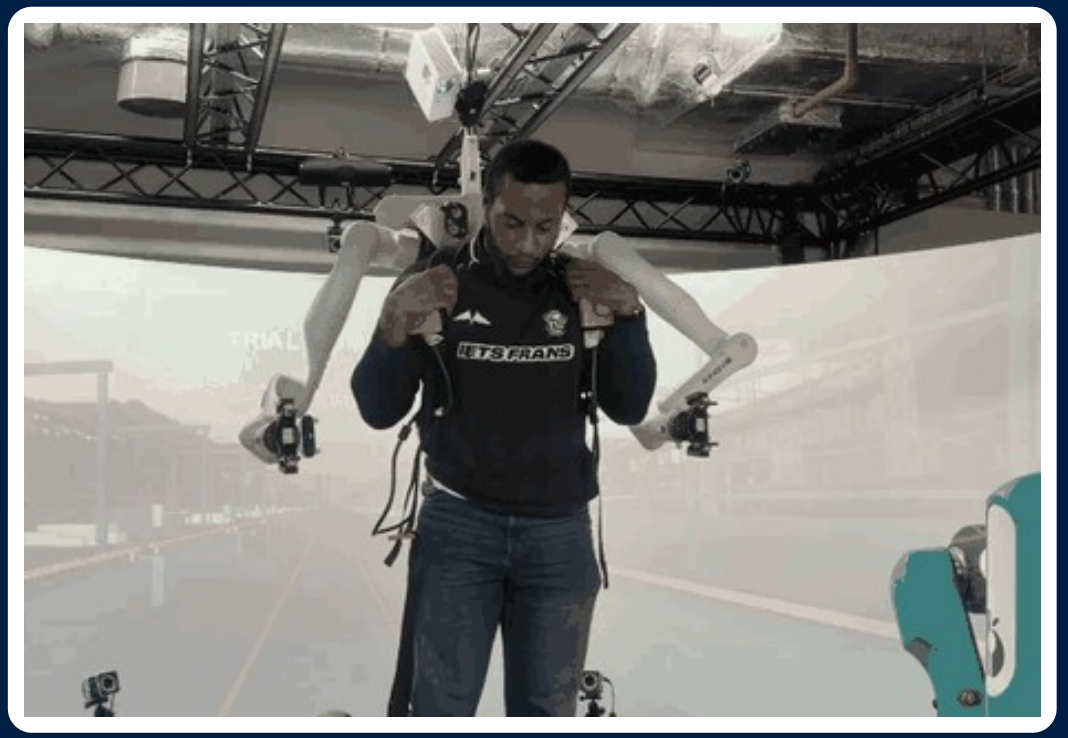
Imperial College London

World-Stable Supernumerary Effectors for Human Augmentation Under Locomotion-Scale Base Motion

Keeping a wearable robot's hands still while the wearer walks

Christian Akabueze
MSc Human and Biological Robotics
Imperial College London · MUVE Lab · 2026

github.com/fechachris4/msc_project



Watch the hands, not the elbows.

Every step moves your extra hands



63–68 mm

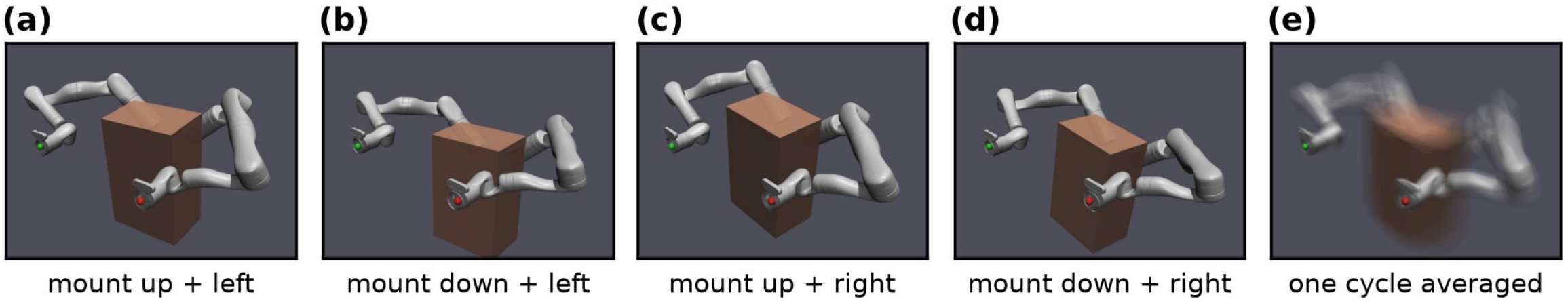
hand motion if the arm were locked (RMS)

~1.8 Hz

how often steps come: fast, repeating motion

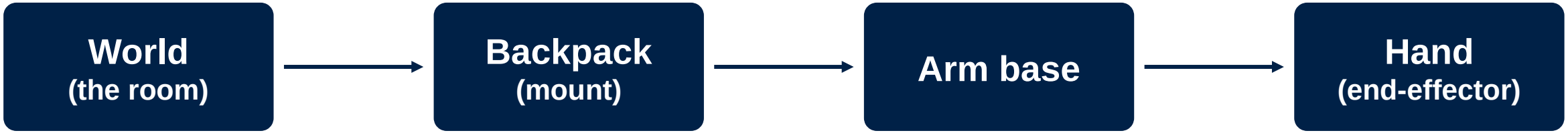
Walking on the treadmill, both arms on.

Goal: fix the hand in the room, not on the body



(e) one cycle averaged: the backpack blurs, the hands stay sharp.
Simulation, controller on. Motion enlarged 3×.

Measure the backpack, cancel it through the arm



Vicon motion capture
measured live

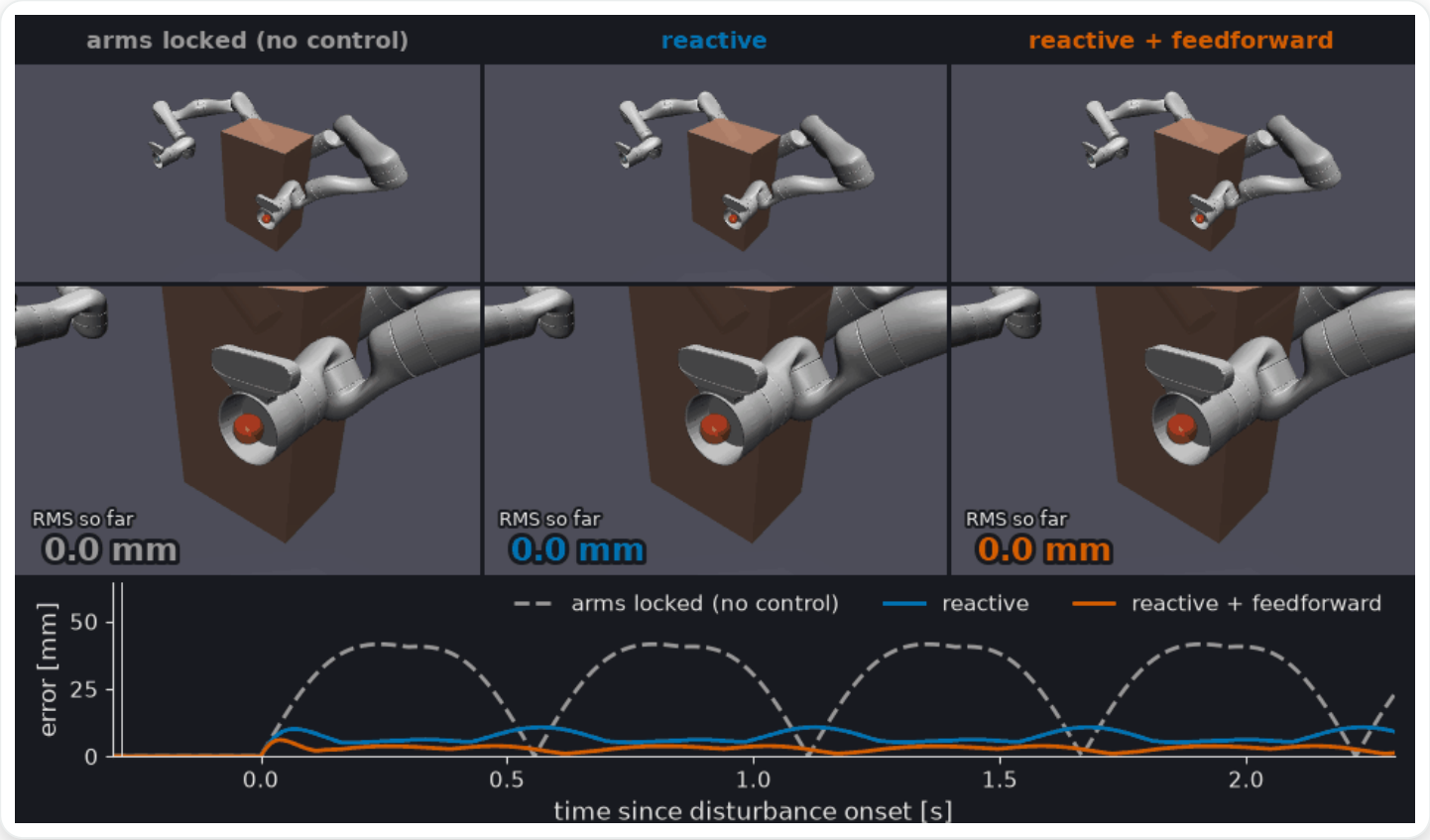
Fixed mount offset
known

Joint angles → kinematics
computed

$$\dot{q} = J^{+}_{\lambda}\,(K_p \cdot e + K_d \cdot \dot{e}) \;+\; (I - J^{+}J)\cdot \dot{q}_0$$

$K_p \cdot e + K_d \cdot \dot{e}$	push the hand back to its spot in the room
J^{+}_{λ}	→ joint speeds, damped near awkward poses
$(I - J^{+}J)\cdot \dot{q}_0$	spare 7th joint drifts to mid-range

Simulation first: my controller in a physics engine



Same motion (1.8 Hz), three controllers. Watch the red target, middle row.

Hand error over 8 s (RMS)

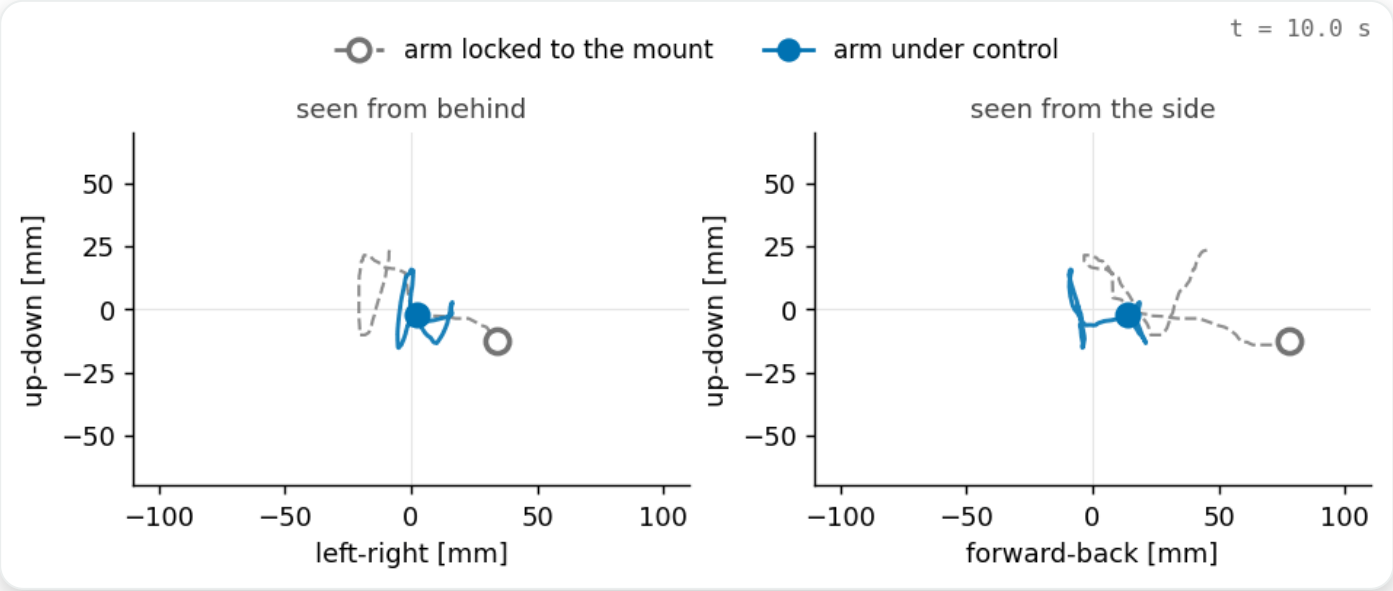
31.0 mm
Arms locked

6.7 mm
Controller

2.6 mm
+ feedforward

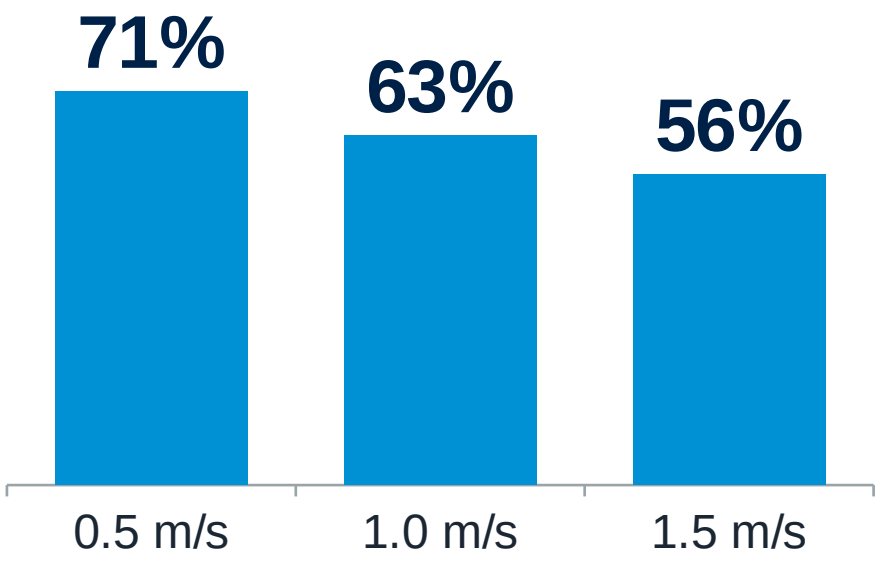
7 people: about two-thirds of walking motion removed

Trials ran the lab's 400 Hz controller (Kalman-filtered Vicon). The analysis is mine.



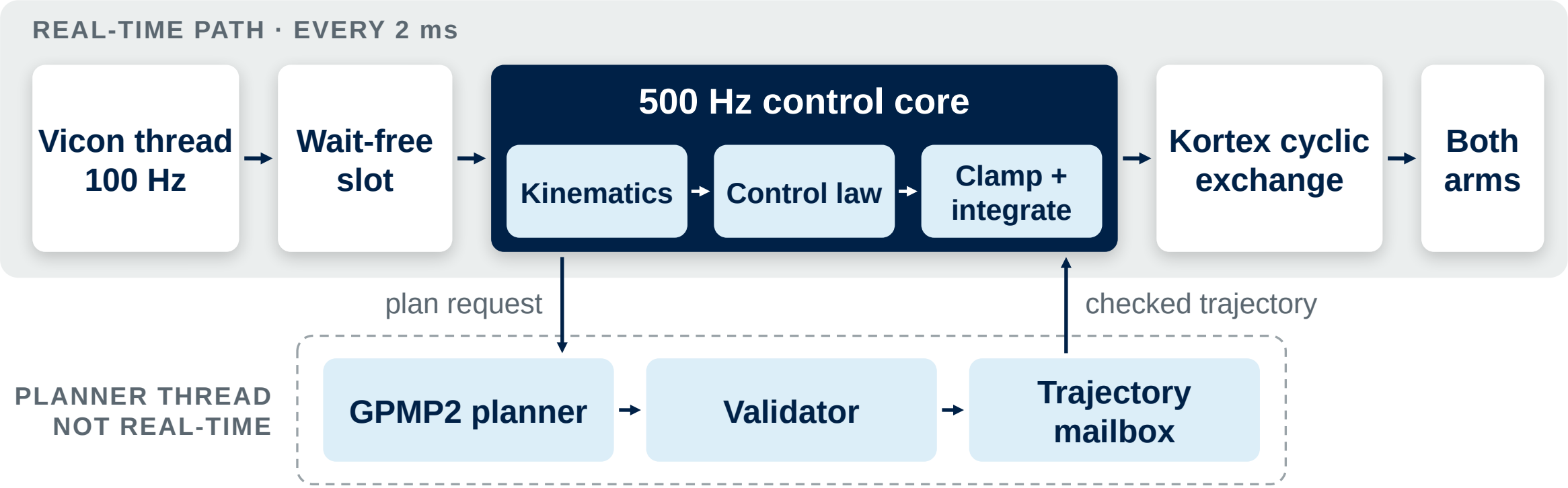
Grey: where the hand would go if the arm were locked. Blue: where it went.

Motion removed, by walking speed



Mean of 6 people (4 at 1.5 m/s) · 128 trials, mostly one arm

Trade-off: planning can never block the 2 ms loop



Gain: the 2 ms loop never waits on Vicon or the planner.
Cost: a new plan starts later, only after it passes the validator.

My C++ controller (HumanSL_MAIN); a MuJoCo twin runs the same core.

Turning up the gain barely helped

38% predicted drop near 1 Hz

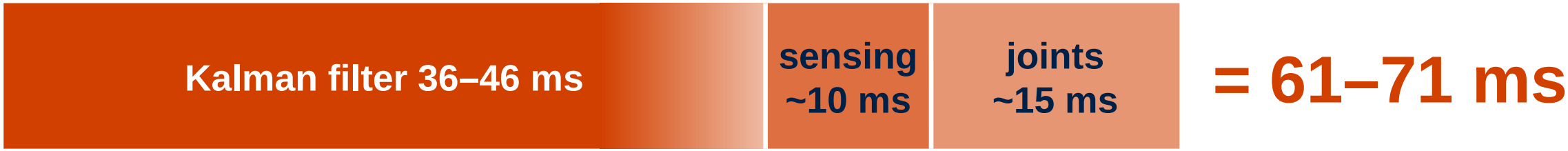
vs

9% measured drop near 1 Hz

Pilot, one wearer: velocity gain Kd raised 0.8 → 2.6

The velocity term was barely acting. Why?

The cause: the velocity arrives 61–71 ms late



share of velocity left uncancelled = 2·sin(*ωd*/2)

ω: step rate · d: delay

Steps at 1.8 Hz, 60 ms late: the velocity is off by **two-thirds** of its own size.

Measured delay, nothing fitted: the model matches the hardware better in 6 of 6 people.

Impact, and what comes next

2/3

of walking motion removed on real arms, worn by 7 people

56–71%, by walking speed

- Next: faster velocity estimate → feedforward on the real arms (sim: 6.7 → 2.6 mm)
- Lesson: measure the latency budget before tuning gains

Thank you. Questions?

Results, simulation, figures

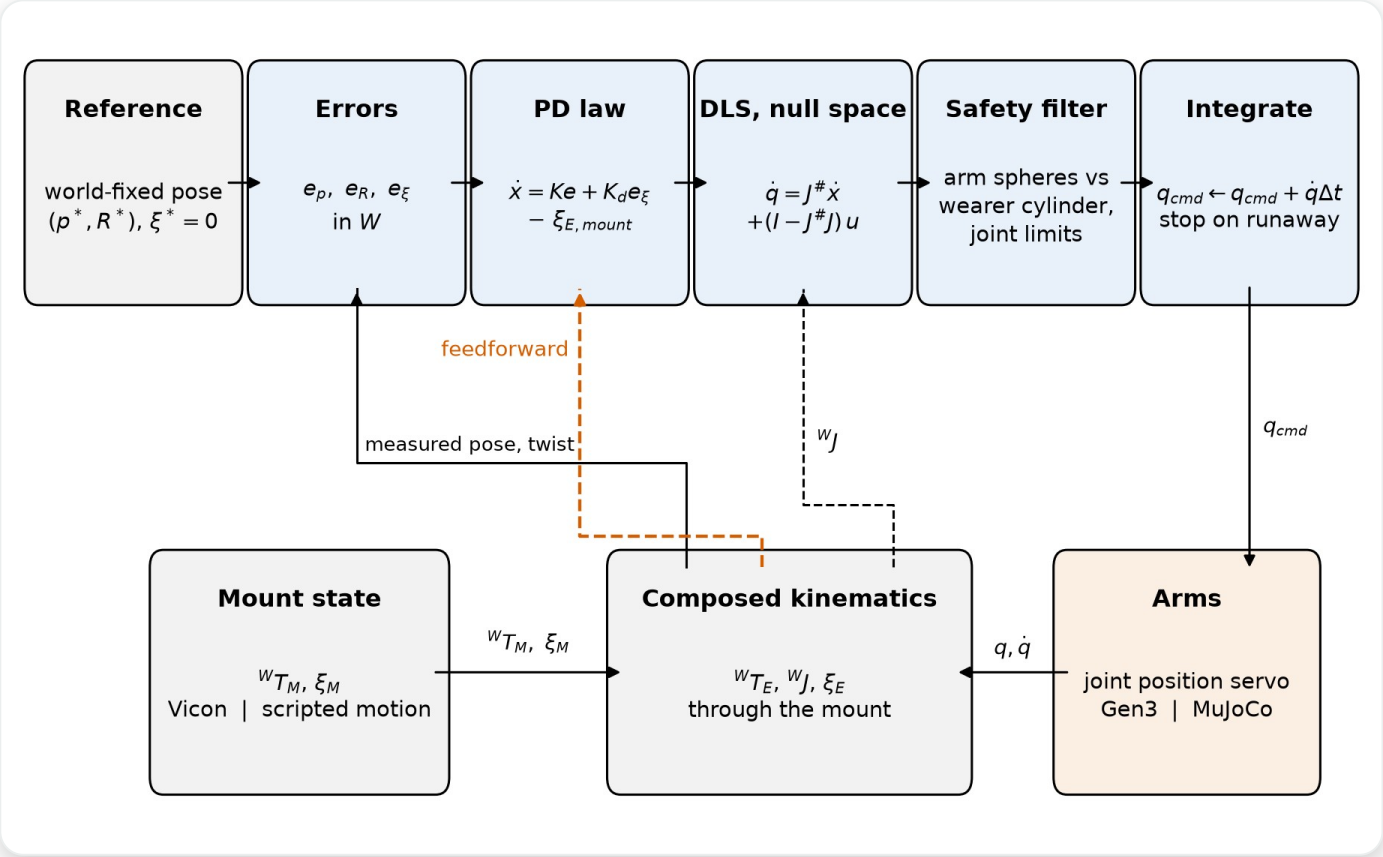
github.com/fechachris4/msc_project

C++ real-time controller

github.com/fechachris4/HumanSL_MAIN

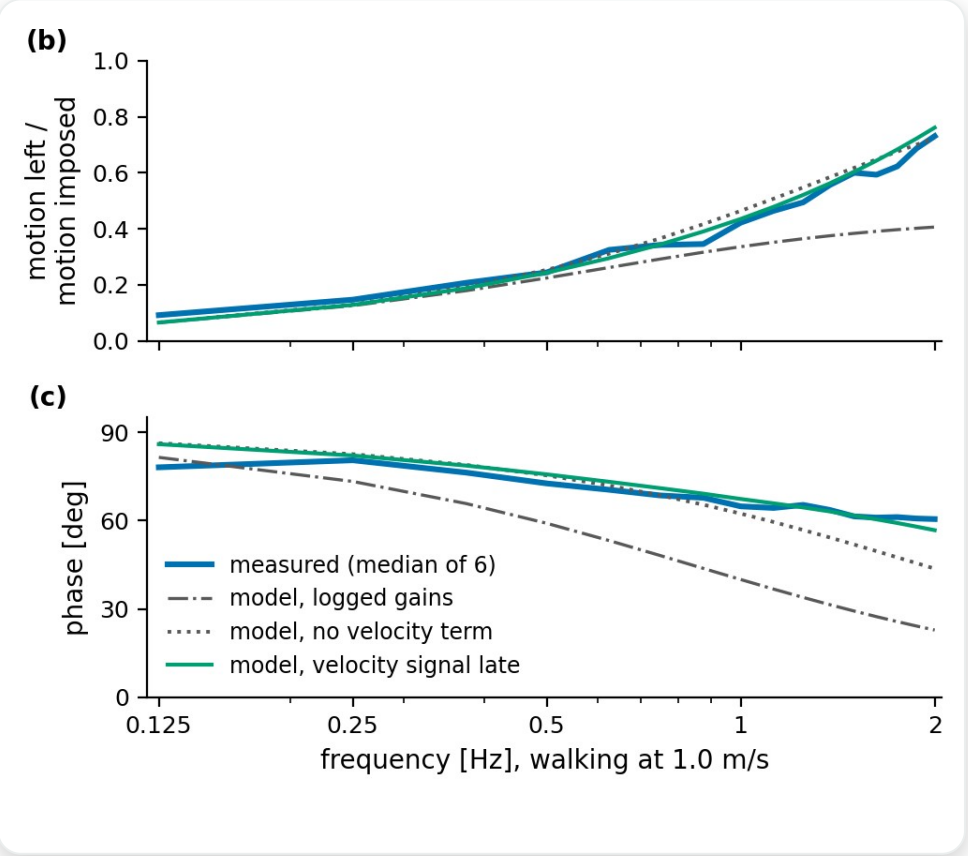
fecha412@gmail.com

Backup: the control loop (simulation)



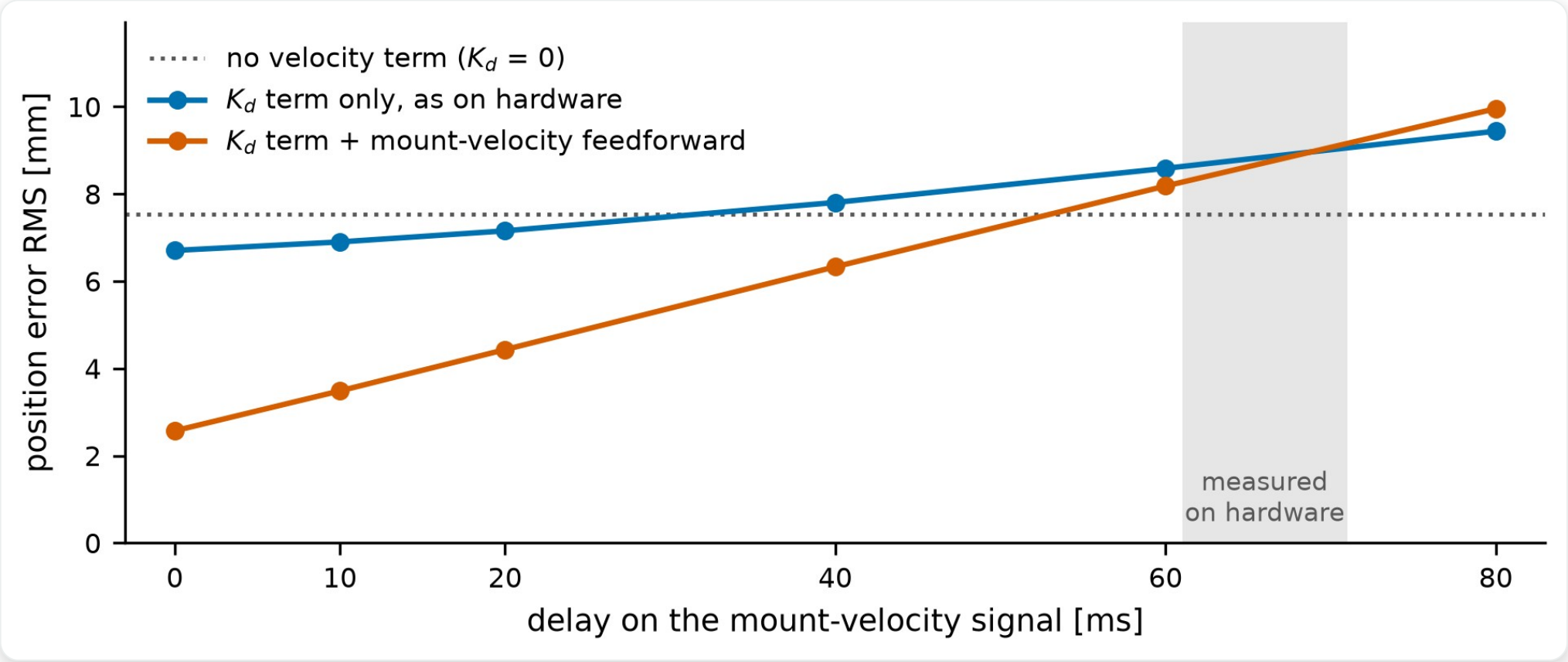
Per cycle, every 2 ms in simulation. Orange dashes: feedforward (simulation only).

Backup: frequency response, 1.0 m/s



Median of 6 people. In amplitude (top), measured follows the no-velocity model (dotted). Green: late-velocity model. Lower = more removed.

Backup: simulation, error vs delay



Velocity signal delayed on purpose, 1.8 Hz. Grey band: the delay measured on hardware.

Backup: limits the data showed

Re-planning	12 of 128 trials: the planner moved the target > 100 mm; those had the most hand motion → a planner setting
Joint speed	at 1.5 m/s a joint at its limit 32% of the time, mostly wrist (~10% at slower walking speeds)
Fixed offset	constant 18–21 mm from the goal follows model-vs-Vicon disagreement → calibration

Backup: pitching and swaying platform

78%

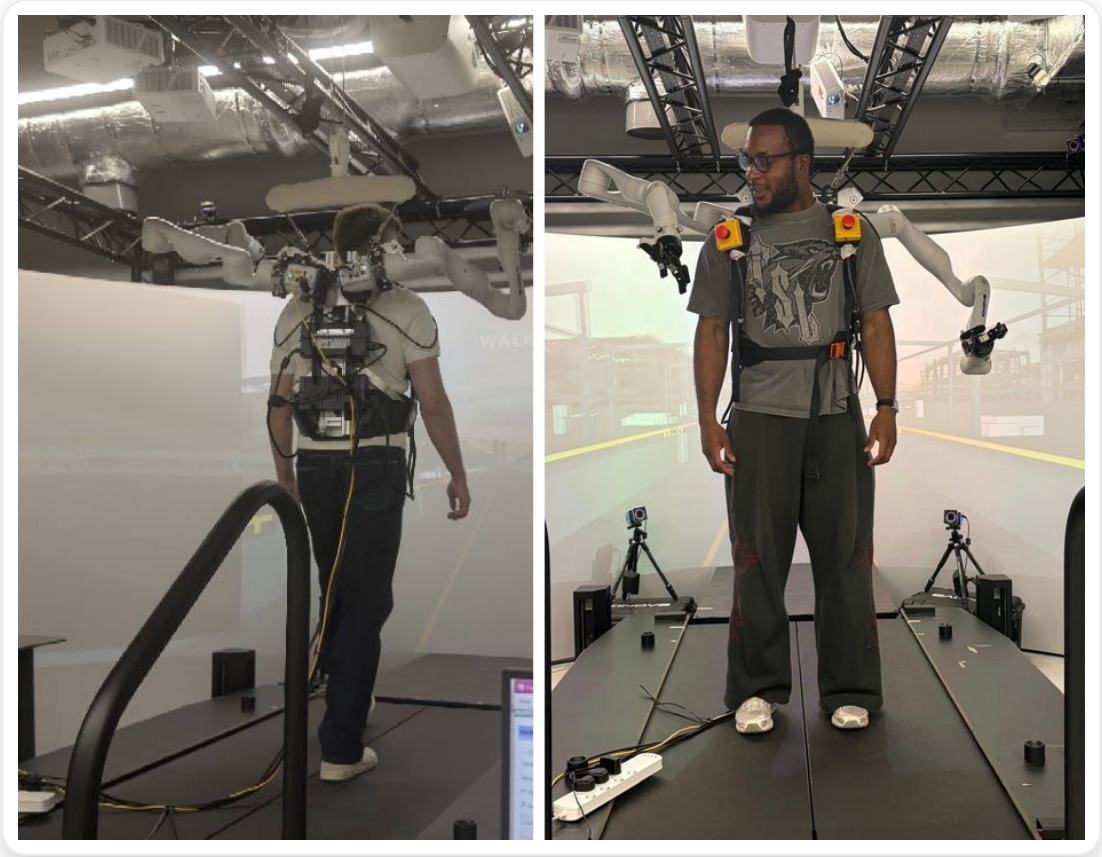
of platform motion removed (5 people)

Locked arm → hand, RMS

Pitch: 94 mm → 14 mm

Sway: 81 mm → 17 mm

Backup: the rig



Two Kinova Gen3 arms on a backpack (mount); Vicon cameras around the treadmill.